

Week 2 — The Learning Loop: How Neural Networks Learn

Generative AI and Deep Learning · Balaji Srinivasan, Ganapathy Krishnamurthi · Lectures 2.1 – 2.3

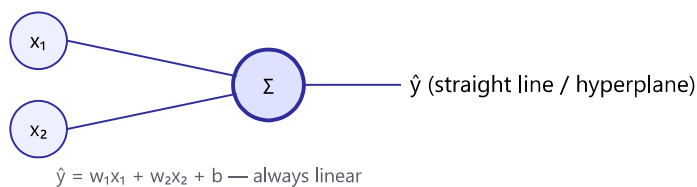
2.1 Universal Approximation Theorem

Recap — Linear Regression vs. a Neural Network

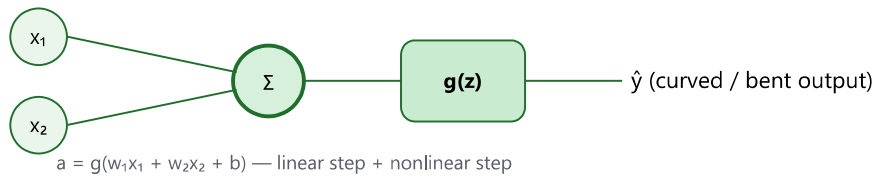
A **linear regression** model takes inputs $x_1, x_2, \dots$, multiplies each by a weight $w_1, w_2, \dots$, sums them up, and produces an output $\hat{y}$. Geometrically this is a **straight line** (or, with more inputs, a **hyperplane**) in the input feature space. No matter how many inputs you have, the relationship between inputs and outputs stays linear.

A **neural network / neuron** does not stop at the weighted sum. Each neuron passes that weighted sum through a **nonlinear activation function** (e.g. sigmoid, tanh). This combination of a *linear operation* followed by a *nonlinear operation* is exactly what makes a neuron strictly more powerful than plain linear regression.

Linear Regression



Neuron (Neural Network building block)



Illustrative diagram (this lecture was presented verbally without a dedicated slide): a linear model can only ever produce a flat/straight relationship, while a neuron bends that relationship through its nonlinearity.

Why Nonlinearity Matters

If you only stack linear operations on top of each other, the result is **still linear** — the composition of linear functions is linear. **Nonlinearity is what allows neural networks to bend, warp and curve the input feature space** so that they can capture complex relationships.

Think of linear regression as fitting a straight line, whereas a neural network can fit curves, surfaces, and very irregular shapes. This is exactly what builds toward the Universal Approximation Theorem: with multiple layers of neurons, each performing a linear operation followed by a nonlinear operation, neural networks can represent increasingly complex functions.

The Theorem (Informal Statement)

Universal Approximation Theorem

A feedforward network with **one hidden layer** and a **nonlinear activation** can approximate **any continuous function on a bounded domain** — given enough neurons in that hidden layer.

"Approximate" means the network can get **arbitrarily close** to the true function. This does *not* mean the network learns this automatically — only that the *capacity* to represent the function exists.

$$\forall f(x) \text{ continuous}, \forall \epsilon > 0, \exists (W, b) \text{ such that } |\text{NN}(x; W, b) - f(x)| < \epsilon$$

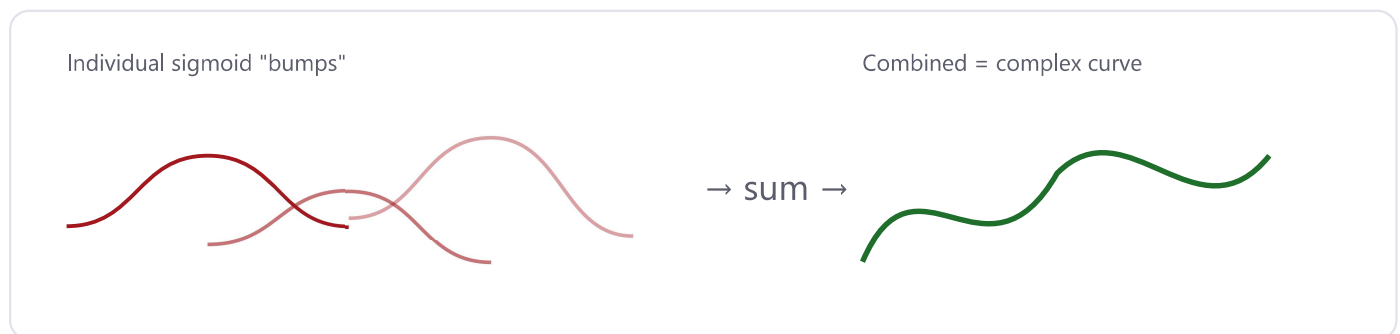
In words: for any continuous function $f(x)$ and any error tolerance $\epsilon > 0$, there *exist* some weights W and bias b such that the difference between the network's output and the true function is smaller than ϵ .

Important caveat

The theorem only guarantees that suitable parameters W, b **exist**. It gives **no guarantee that you will actually be able to find them** through training.

Intuition — Building Curves out of Bumps

Nonlinear activations create "bumps" or "waves" in function space. By stacking and combining these bumps, a network can approximate almost any shape. For example, a sigmoid can be used as a building block for piecewise approximation — many small piecewise sigmoid "steps" combine to trace out a complex curve.



Illustrative sketch of the "stacked bumps" intuition described in the lecture.

Key Takeaways

- Linear regression is limited to straight lines / hyperplanes — for both regression *and* classification, the best a linear model can do is draw a straight line or fit a hyperplane decision boundary. If the true relationship or decision boundary is wavy, wiggly, or otherwise complex, linear models simply won't work.
- Neural networks with nonlinearity are **flexible universal approximators** — they can be used to approximate pretty much any function.
- In theory, **one hidden layer is enough**. In practice, deeper networks are used because they are more efficient (fewer total neurons/parameters needed for the same expressive power).
- The Universal Approximation Theorem explains the **capacity / power** of neural networks — **not their trainability**. There is a real difference between representational capacity, trainability, and generalizability.
- Even if a network is trained successfully on the data you gave it, there is **no guarantee it will generalize to unseen data**.

What is the Forward Pass?

The forward pass is the process of taking an input X and feeding it as input to the first layer, taking the output of that first layer and feeding it to the next layer, and so on all the way to the output prediction $\hat{y}$.

The input X is assigned as $a^{[0]}$ (the "activation of layer 0"). For the first layer, we do the affine transform $z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$, then apply a pointwise nonlinearity g to get the output $a^{[1]}$. This activation is then fed into the next layer, and the same computation repeats all the way to the final output layer L , where the layer's output $a^{[L]}$ is $\hat{y}$.

The Forward Pass: From Input to Prediction

The forward pass is the process of taking an input vector $\mathbf{x}$ and feeding it through the network, layer by layer, to produce a final prediction $\hat{\mathbf{y}}$.

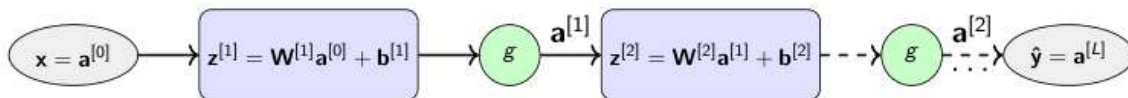


Figure: Information flows sequentially through linear layers and non-linear activations.

The computation for every layer follows the same two-step, vectorized pattern:

1. **Linear Combination:** Calculate the weighted sum for all neurons in the layer.

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

2. **Non-linear Activation:** Apply the activation function element-wise.

$$\mathbf{a}^{[l]} = g(\mathbf{z}^{[l]})$$

The final activation vector $\mathbf{a}^{[L]}$ is the network's prediction $\hat{\mathbf{y}}$.

The forward pass — information flows sequentially through linear layers and nonlinear activations.

The Two-Step Vectorized Pattern

Every layer's computation follows the **same two-step, vectorized pattern**. If the input X is a vector, then z is computed as a matrix (weights) times a vector (previous activations) plus a vector (bias) — the same computation repeats through all the layers.

1. **Linear combination** — calculate the weighted sum for every neuron in the layer:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$z^{[l]}$, the pre-activation of the current layer, equals the weights of the current layer times the activations from the previous layer, plus the bias of the current layer.

2. **Non-linear activation** — apply the activation function *element-wise*, since $z^{[l]}$ is a vector and g acts on its individual elements:

$$a^{[l]} = g(z^{[l]})$$

The final activation vector $a^{[L]}$ is exactly the network's prediction $\hat{y}$.

Loss Functions: Quantifying the Error

A loss function has to be computed to quantify the error. It takes the true label y and the prediction $\hat{y}$ and returns a **single scalar number** representing the error. This error is exactly what we aim to minimize in order to optimize the parameters of the network — the weights and biases of the weight matrices seen above.

For Regression Tasks — Mean Squared Error (MSE)

Loss Functions: Quantifying the Error

A loss function $\mathcal{L}(y, \hat{y})$ takes the true label (y) and the prediction ($\hat{y}$) and returns a single number representing the error. This error is what we aim to minimize during training.

For Regression Tasks

Mean Squared Error (MSE)

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- Calculates the average squared difference between true and predicted values.
- Penalizes large errors very heavily.

MSE is nothing but the average squared difference between the true and predicted values, over all data points. It penalizes large errors heavily. The downside: because errors are squared, just a few outliers can cause a very high error value, which can be misleading about overall model performance.

For Classification Tasks — Cross-Entropy Loss

Loss Functions: Quantifying the Error

For Classification Tasks

Cross-Entropy Loss

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

- y_c is 1 if c is the true class, 0 otherwise.
- $\hat{y}_c$ is the predicted probability for class c .
- Penalizes the model for being confident and wrong.

Example (3 classes):

- **True:** Cat $\rightarrow [1, 0, 0]$
- **Good Pred:** $[0.8, 0.1, 0.1] \rightarrow$ Low Loss
- **Bad Pred:** $[0.1, 0.2, 0.7] \rightarrow$ High Loss

The summation runs over the total number of classes C (e.g. if you're classifying an input among C categories). Here y_c is the ground truth — it is **1** for the correct class and **0** for every other class (a one-hot vector), and $\hat{y}_c$ is the predicted probability for class c .

Cross-entropy **penalizes the model for being confident and wrong**. For example, in a three-class problem where "cat" is the first class, the ground truth one-hot vector is $[1, 0, 0]$.

- **Good prediction:** a very high probability for the correct class and very low probabilities for the other classes (e.g. $[0.8, 0.1, 0.1] \rightarrow$ low loss).
- **Bad prediction:** a high probability assigned to the wrong class and a low value for the correct class $\rightarrow$ high loss (a wrong, over-confident prediction).

Constraint to remember

In both the good and bad cases, the output probabilities must always add up to **1**. How this is enforced (softmax) is examined in more detail in a later lecture.

This lecture was intentionally a "bird's eye view" of what the forward pass entails — the next lecture covers gradient descent, i.e. how the network actually uses this error to learn.

The Intuition — A Hiker on a Foggy Mountain

Gradient descent is the algorithm that helps a neural network learn. Imagine you are on a mountainside in thick fog, trying to find your way to the bottom of the mountain — the lowest valley. At every step you ask two simple questions:

1. **Which way is downhill?** You feel the slope under your feet. The direction of steepest *ascent* is the **gradient**. So you take a step in the **opposite** direction.
2. **How big a step should you take?** This is your **step size**, or the **learning rate**. Too small and it will take a long time to reach the bottom; too large and you might overshoot.

Gradient Descent: The Intuition

The Analogy: A Hiker on a Foggy Mountain

Imagine you are on a mountainside in thick fog and want to get to the lowest valley. What do you do?

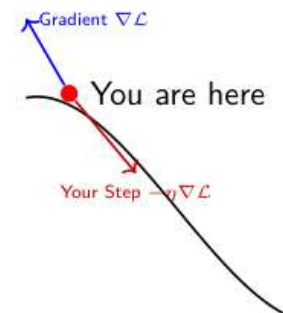
You ask two simple questions at every step:

1. Which way is downhill?

- You feel the slope under your feet.
- The direction of steepest ascent is the **Gradient** ($\nabla \mathcal{L}$).
- So, you take a step in the **opposite** direction.

2. How big a step should I take?

- This is your "step size" or **Learning Rate** (η).
- Too small: takes forever. Too big: overshoots the valley.



Why Gradient Descent? Navigating the Loss Landscape

The goal is to find the parameters θ that minimize the loss function $\mathcal{L}(\theta)$ — i.e. to find the lowest point in the loss landscape. At each step we move in the direction of the steepest descent:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla \mathcal{L}(y, f(x; \theta_{\text{old}}))$$

It can be proven mathematically that the gradient is the direction of steepest *ascent* — so a step in the direction *opposite* to it is the direction of steepest *descent*.

The catch: loss functions that arise in neural networks (as a function of the network's parameters) tend to have **multiple minima**, and there is no guarantee that gradient descent will end up at the **global minimum**.

- **Global minimum** — the true lowest point in the function; the optimal solution.
- **Local minimum** — just a "valley" that traps the algorithm, preventing it from reaching the global minimum. This happens quite often in neural network training.

Why Gradient Descent? Navigating the Loss Landscape

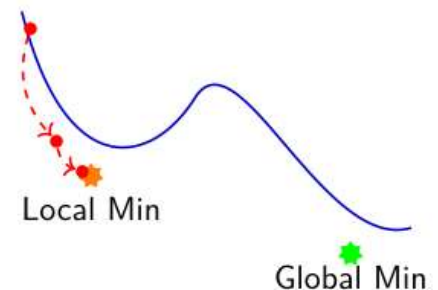
The Goal

Find the parameters θ that minimize the Loss Function $\mathcal{L}(y, f(x; \theta))$. This means finding the lowest point in the landscape.

The Update Rule

At each step, we move in the direction of steepest descent:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla \mathcal{L}(y, f(x; \theta_{\text{old}}))$$



The Challenge: Local vs. Global Minima

- **Global Minimum:** The true lowest point; the optimal solution.
- **Local Minimum:** A "valley" that traps the algorithm, preventing it from reaching the global minimum.

The algorithm can get trapped in a local minimum instead of reaching the true global minimum.

Gradient Descent in Action — Linear Regression

Recall the linear regression model coded earlier: the best-fit line is $\hat{y} = mx + c$, with parameters m and c . The loss function is the mean squared error, $(y - \hat{y})^2$ averaged over the data. The update rule is exactly the gradient descent step applied to each parameter — find the derivative of the loss with respect to each parameter, then take a small step opposite to the direction in which the function increases:

$$m := m - \eta \frac{\partial \mathcal{L}}{\partial m} \quad c := c - \eta \frac{\partial \mathcal{L}}{\partial c}$$

GD in Action: Linear Regression

1. The Model

Find the best line $\hat{y} = mx + c$. Parameters are $\theta = \{m, c\}$.

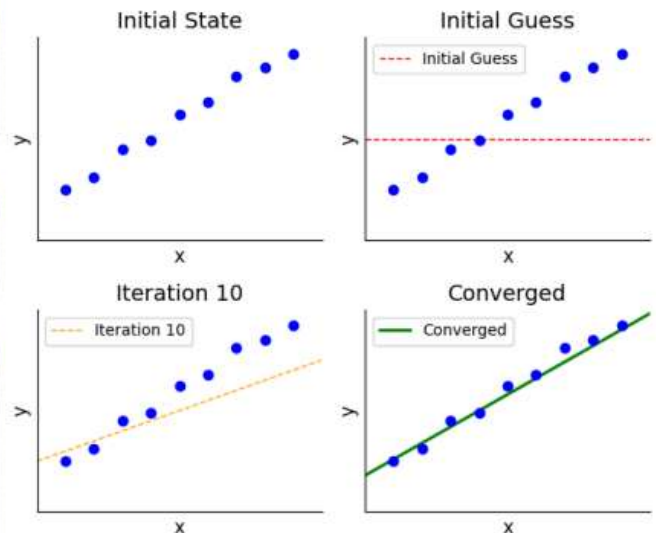
2. The Loss Function (MSE)

$$\mathcal{L}(m, c) = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$$

3. The Update Rules

$$m := m - \eta \frac{\partial \mathcal{L}}{\partial m}$$

$$c := c - \eta \frac{\partial \mathcal{L}}{\partial c}$$



Starting from an initial guess (e.g. the average of y across all points), by iteration 10 the line is already approaching the data, and after roughly 20–30 iterations it has converged. Note that the converged line will **not** pass through every point exactly — there is always noise in real data, so small residual errors remain. What gradient descent does is get rid of the *large* errors.

The Three Variants of Gradient Descent

There are many gradient descent algorithms, each improving on the others for faster convergence, but at their core there are only three fundamental variants, distinguished by how much data is used to compute each update step:

Aspect	Batch GD	Stochastic GD (SGD)	Mini-Batch GD
Data used per step	Entire dataset	One single data point	A subset (mini-batch) of the data

Aspect	Batch GD	Stochastic GD (SGD)	Mini-Batch GD
Update frequency	Once per epoch	After each data point	After each mini-batch
Convergence speed	Slow	Fast per iteration	Moderate
Convergence stability	Smooth and stable	Noisy and erratic	Relatively smooth
Computational efficiency	Very inefficient	Efficient (one point at a time)	Most efficient — fits in GPU memory
Memory requirement	Very high	Very low	Moderate

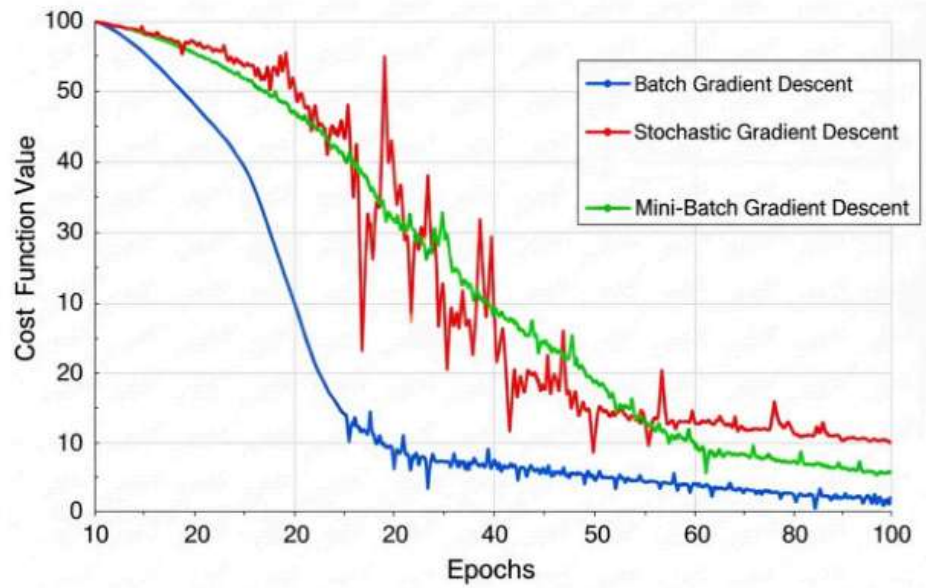
An **epoch** is one full pass through all the data points. Batch GD needs the entire dataset loaded to take just one step, which is why its memory requirement is so high and it is computationally inefficient for large datasets. SGD looks at only one point at a time, so it's cheap per step but its path to the minimum is noisy. Mini-batch GD (recall: 8 points at a time in the earlier linear regression implementation) loads a subset of data into the GPU's virtual memory and processes it there — this is why it is the most efficient option computationally.

The winner: Mini-Batch Gradient Descent

Mini-batch gradient descent is preferred for essentially all deep learning applications. It offers the best compromise between memory and computational efficiency, since batches of data (for both the forward pass and backpropagation) can be transferred into the GPU's virtual RAM for fast computation.

Convergence Behaviour, Visualized

Convergence of Gradient Descent Methods



Plotting the loss function against epochs: Batch GD (blue) is very smooth; SGD (red) is noisy/erratic; Mini-Batch GD (green) is a compromise between the two.